AGV KIM2A — System Reference

Oct 22, 2026 · @Jo'el Evander Nathanael Irawan

Overview

KIM2A (profile `agv1_kim`) is a differential-drive magnetic-tape AGV run by a Python asyncio controller with a Flask web HMI on its Ubuntu PC. The code lives at github.com/joelevander10/agv-kim2a-controller, branch `main`, and on the AGV in `~/agv-kim2a-controller`.

Item	Value
AGV PC login	`agv1-kim` (SSH)
AGV PC address (AGV network)	192.168.2.100
AGV PC address seen on Wi-Fi	192.168.0.102 (DHCP — can change)
Web HMI	http://<AGV address>:5000
DIO module (DI + DO, Modbus TCP)	192.168.1.31:502, unit 1
AO module (wheel speed, Modbus TCP)	192.168.1.30:502, unit 1
RFID reader (TCP)	192.168.5.10:2022
PLC (Mitsubishi SLMP / MC binary)	192.168.4.10:1280
Magnetic sensor	Roboteq MGS1600 on CAN, node 6, COB-ID 390 (0x186)
Calibration encoder	CANopen friction wheel, node 1, 125 kbit/s
Drive wheels	0.18 m diameter, gearbox 1:30, track 0.46 m
Front sensor ahead of axle	0.076 m (placeholder — measure and update)
USB keypad	name contains "XING WEI"

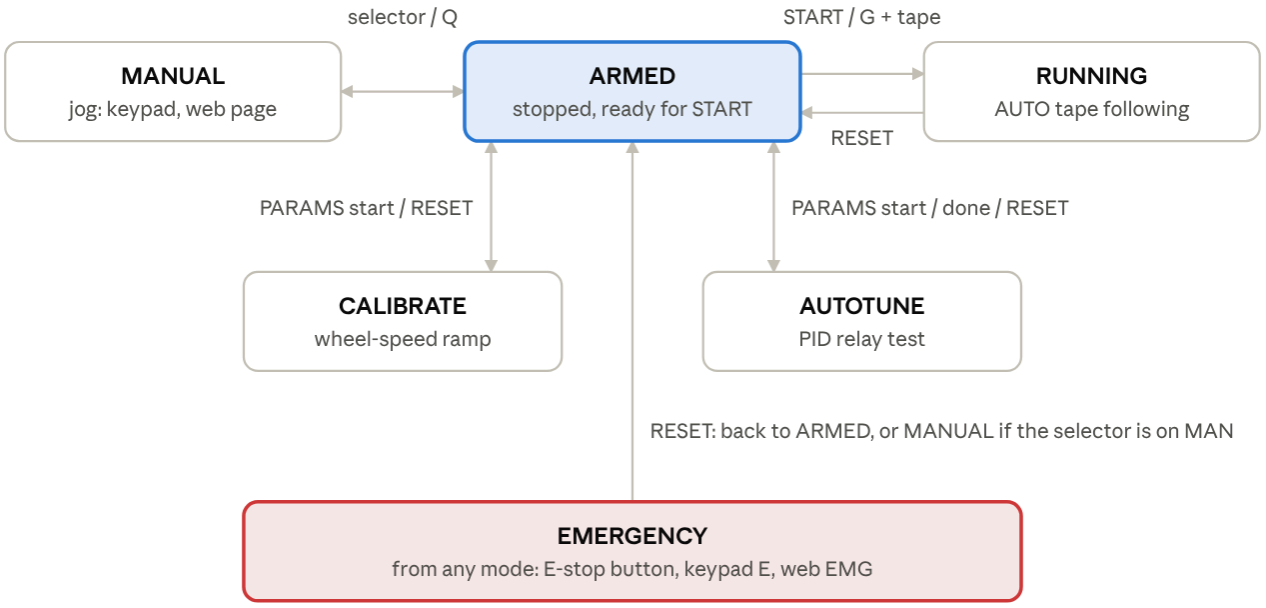
Run it: `cd ~/agv-kim2a-controller && AGV_ID=agv1_kim python3 main.py`. The profile is `profiles/agv1_kim.json`; `agv2_kim.json` currently fails to load (missing `DECEL_RATE`).

Main files: `main.py` (start-up, drivers), `modes.py` (mode manager, AUTO and MANUAL loops), `config.py` (loads the profile), `core/pid.py`, `core/sequence_engine.py` (RFID sequences), `keyboard_control.py` + `drivers/keypad_evdev.py` (keypad), `autotune.py`, `app/app.py` (web HMI).

Operating modes

The AGV always starts in ARMED; from there the selector or Q goes to MANUAL, START (with tape under the sensor) goes to RUNNING, and RESET brings every run back to ARMED.

ARMED is the hub: every run starts and ends there



operating modes · 6 states

- **Emergency** stops everything with the motors idled (brakes released, the AGV can be pushed by hand). It latches until RESET, even after the button is released.
- **MANUAL brakes:** whenever nothing is jogging (no keypad move key or web jog button held), both wheel brakes (DO2, DO5) are ON so the AGV holds still; they release on the next jog command.
- **ROTATING (180° turn):** T / Y or the ROTATE buttons on the MANUAL page start it from MANUAL or ARMED with tape under the sensor. The AGV spins on the spot at 0.05 m/s, loses the tape at about 45°, finds it again from about 135°, creeps at 0.02 m/s and stops when the sensor is within 5 mm of centre, then returns to the mode it came from. T / Y again, STOP, RESET, moving the selector or emergency stop it; it gives up after 260° or 40 s. Use it on straight tape away from forks and RFID tags. Settings: profile block `rotate`.

- **Q override:** Q forces MANUAL or AUTO until someone moves the physical selector; then the selector rules again.
- **Web button mode** (MANUAL page) replaces the panel START / RESET / selector with on-screen buttons; the physical EMERGENCY still works.
- **System error** (a driver stops answering: DI module, CAN or RFID) brakes the AGV and holds until the driver comes back.
- **PLC status:** MANUAL sets M2003, ARMED/RUNNING set M2004, RUNNING sets M2005, emergency sets M2001/M2002.

IO mapping

The AGV uses 5 of 16 digital inputs (DI4–DI7 are reserved for a pendant that is not fitted), 6 of 16 digital outputs and both analog outputs; DI9–DI15 and DO6–DO15 are free. Channel numbers come from `io_mapping` and `motor_channels` in the profile.

Digital inputs (DIO module 192.168.1.31, `DI_FLIPPED` = 0)

DI	Signal	Contact / logic
0	START	Momentary push button, rising edge starts AUTO from ARMED
1	RESET	Momentary push button, rising edge = reset
2	MODE selector	ON = MANUAL, OFF = AUTO
3	EMERGENCY	ON = emergency active (NO contact)
4–7	Pendant FWD / REV / LEFT / RIGHT	Not used — no pendant fitted (`io_mapping.PENDANT_ENABLED` = false); set true to jog from these inputs again
8	PROX station marker	Magnetic proximity; rising edge stops the AGV at a trolley station (`marker_side: prox`)

Digital outputs (DIO module 192.168.1.31)

DO	Signal
0	Left motor FORWARD
1	Left motor REVERSE
2	Left motor BRAKE

DO	Signal
3	Right motor FORWARD
4	Right motor REVERSE
5	Right motor BRAKE

Motor driver alarm inputs are not wired (`do_alarm` = 888, disabled).

Analog outputs (AO module 192.168.1.30, register base 4096)

AO	Signal	Range
0	Left wheel speed	0–5 V commanded (module range 10 V, 12-bit)
1	Right wheel speed	0–5 V commanded (module range 10 V, 12-bit)

Motor map (corrected 2026-10-26): left rpm = 647.5 × V − 177.1, right rpm = 644.9 × V − 149.55 (motor rpm, before the 1:30 gearbox). The 2026-05-30 encoder calibration over-read speed 2×, so check the encoder's `COUNTS_PER_REV` and `WHEEL_DIAMETER_M` before running the wheel calibration again. At the 5 V limit the AGV tops out at about 0.96 m/s.

CAN bus (CANable adapter): MGS1600 magnetic sensor node 6 with flags tape detect = 2, left marker = 4, right marker = 8, sensor fail = 128; sensor timeout 0.5 s. The calibration encoder shares the adapter and is only used during wheel-speed calibration.

USB keypad

The keypad uses the AGV Kayaba layout; keys for features KIM2A does not have are refused with a log message and do nothing. Hold keys act only while held; one-press keys act on the press.

Key	Action	Type	Works in
W / ↑	Forward	Hold	MANUAL
S / ↓	Reverse	Hold	MANUAL
A / ←	Spin left on the spot	Hold	MANUAL
D / →	Spin right on the spot	Hold	MANUAL
W + A / W + D	Curve forward left / right without stopping (inner wheel at MANUAL SLOW speed)	Hold	MANUAL

Key	Action	Type	Works in
S + A / S + D	Curve backwards left / right	Hold	MANUAL
G	START AUTO (same as the START button; needs tape under the sensor)	Press	ARMED
T / Y	Rotate 180° on the tape LEFT (CCW) / RIGHT (CW); press T or Y again to stop the turn	Press	MANUAL, ARMED
1	AUTO speed APPROACH (0.09 m/s)	Press	ARMED, RUNNING
2	AUTO speed SLOW (0.09 m/s)	Press	ARMED, RUNNING
3	AUTO speed MEDIUM (0.5 m/s)	Press	ARMED, RUNNING
4	AUTO speed HIGH (0.8 m/s)	Press	ARMED, RUNNING
5	AUTO speed MAX (1.0 m/s)	Press	ARMED, RUNNING
Q	Switch MANUAL ↔ AUTO (overrides the selector until it is moved)	Press	Any
K / L	Follow left / right tape track	Press	ARMED, RUNNING
E	EMERGENCY STOP, latched until RESET	Press	Any
ENTER	RESET (same as the RESET button)	Press	Any
Z	PLC pusher EXTEND (M2212)	Hold	MANUAL, SLMP on
X	PLC pusher RETRACT (M2213)	Hold	MANUAL, SLMP on
U / I / O	PLC top conveyor FWD / RVS / CLR ERR (M2104 / M2105 / M2106)	Hold	MANUAL, SLMP on
B / N / M	PLC bottom conveyor FWD / RVS / CLR ERR (M2124 / M2125 / M2126)	Hold	MANUAL, SLMP on
F1 … F11 / F12	Select CABANG 1 … 11 (same as Pilih Cabang on HOME, kept through RESET); F12 clears it. 7 / 8 / 9 / 0 are not used	Press	Any

Key	Action	Type	Works in
R	Auto reverse — not on KIM2A	—	—

- Driving stops within 0.4 s if the keypad is unplugged or the controller stops hearing it; once stopped in MANUAL the wheel brakes come on.
- PLC bits drop to 0 on release, unplug, or when the AGV leaves MANUAL; opposite keys held together (Z+X, U+I, B+N) cancel.
- Settings: profile block `keypad` — `ENABLED`, `NAME_CONTAINS` ("XING WEI"), `GRAB` (keys go only to the AGV).
- Setup on the AGV PC: `sudo apt install python3-evdev`, `sudo usermod -aG input agv1-kim`, reboot; check with `python3 debugging/list_keypads.py`.

PLC (SLMP) mapping

The AGV talks to the trolley PLC at 192.168.4.10:1280 every 20 ms over SLMP (MC protocol, binary); it writes its status and requests and reads the PLC's inputs and lamps. R = AGV reads, R/W = AGV writes.

AGV status and requests

Device	Meaning	Dir
M0	PB START AUTO	R
M1	MODE: MAN	R
M2	MODE: AUTO	R
M3	EMERGENCY	R
M4	MASTER ON	R
M2000	AGV.READY (not starting up, not emergency)	R/W
M2001	AGV.EMERGENCY	R/W
M2002	AGV.REQ TROLLEY EMERGENCY (follows AGV emergency)	R/W
M2003	AGV.MANUAL	R/W
M2004	AGV.AUTO (ARMED or RUNNING)	R/W
M2005	AGV.RUNNING	R/W
M2010–M2018	AGV.REQ.SEQUENCE.0–8 (1 s pulse)	R/W

Device	Meaning	Dir
M2040–M2048	SEQUENCE.0–8.COMPLETE	R

Trolley manual control (MANUAL mode only; held while the button or key is held)

Device	Meaning	Keypad	Dir
M2212	Pusher EXTEND	Z	R/W
M2213	Pusher RETRACT	X	R/W
M2104	Top conveyor FWD	U	R/W
M2105	Top conveyor RVS	I	R/W
M2106	Top conveyor CLR ERR	O	R/W
M2107	Top conveyor BRAKE	—	R/W
M2108	Top conveyor BRAKE LATCH	—	R
M2109	Top conveyor BRAKE RELEASE	—	R/W
M2124	Bottom conveyor FWD	B	R/W
M2125	Bottom conveyor RVS	N	R/W
M2126	Bottom conveyor CLR ERR	M	R/W
M2127	Bottom conveyor BRAKE	—	R/W
M2128	Bottom conveyor BRAKE LATCH	—	R
M2129	Bottom conveyor BRAKE RELEASE	—	R/W

PLC outputs (Y) and inputs (X) — read only, shown as lamps on the TROLLEY page

Device	Meaning
Y0 / Y1 / Y2 / Y3	Top conveyor FWD / RVS / CLEAR ERROR / BRAKE
Y4 / Y5 / Y6 / Y7	Bottom conveyor FWD / RVS / CLEAR ERROR / BRAKE
Y10 / Y11	Bottom pusher RED (+) / RED (−)
Y12 / Y13	Bottom pusher BLACK (−) / BLACK (+)
Y14 / Y15	Top pusher RED (+) / RED (−)

Device	Meaning
Y16 / Y17	Top pusher BLACK (−) / BLACK (+)
Y20	PL READY
X0	PB START AUTO
X1	SS MAN/AUTO
X2	EPB TROLLEY
X3	Spare
X4 / X5 / X6 / X7	Proximity TOP front-right / rear-right / rear-left / front-left
X10 / X11 / X12 / X13	Proximity BOTTOM front-right / rear-right / rear-left / front-left

Speeds and PID

AUTO has five speed modes — MAX 1.0, HIGH 0.8, MEDIUM 0.5, SLOW 0.09 and APPROACH 0.09 m/s — each with its own PD gains. Since the 2026-10-26 motor-map fix these are **real** speeds (before it the AGV drove at half the set value). MAX needs slightly more than the 5 V output limit, so its real top is about 0.96 m/s; both wheels are slowed together there so steering is kept. MEDIUM and MAX start as a copy of HIGH until auto-tuned; values saved from the PARAMS page override these profile values.

Speed mode	Target (m/s, real)	KP	TD	N	V_RED_COEF	Output clamp (rpm)
MAX	1.00 (≈ 0.96 at 5 V)	1.15	0.30	20	1.8	±650
HIGH	0.80	1.15	0.30	20	1.8	±650
MEDIUM	0.50	1.15	0.30	20	1.8	±650
SLOW (corners)	0.09	1.3	0.16	12	1.0	±650
APPROACH (station, guide bar on the right)	0.09	0.225	0.20	10	0.2	+4 into bar / −40 away

Shared setting	Value
Manual HIGH / SLOW speed	0.5 / 0.25 m/s

Shared setting	Value
Acceleration	0.025 m/s² toward SLOW / APPROACH; split: ACCEL_RATE_MEDIUM / _HIGH / _MAX (each 0.025 m/s² until changed on PARAMS)
Deceleration / APPROACH deceleration	0.15 / 0.02 m/s² (all ramps are real m/s² since 2026-10-27; before, each ~10 Hz sensor frame stepped as if 10 ms had passed, so the old values were 10× the real rate — saved values were divided by 10, same behaviour)
Integral TI / deadband / max	25 / 20 mm / 400
Control period DT	0.01 s
Speed-reduction filter / cap	0.15 / 45 %
Sensor limits	reject |pv| > 100 mm, clamp steps > 40 mm
Corner feedforward	on in SLOW; only in a corner zone set by a tag (set_speed SLOW with "corner": "left" / "right", optional "radius_m"; true = profile direction), radius 3.5 m (measured from the run logs; was 1.5 m), scale 0.85, filter 0.15 in / 0.6 out

2026-10-26 remap: every speed, ramp and rpm-based value (KP, V_RED_COEF, clamps) was halved together with the motor map, so the AGV behaves exactly as before; only HIGH, MEDIUM and MAX now run at the real speeds asked for. TD and N are unchanged. Saved PARAMS values from before the fix are converted automatically on the first start.

The error is e = −(sensor offset), so positive PID output speeds the left wheel and turns right. The tape followed is the LEFT track by default; RFID `set_track` tags or keys K/L switch it. The total steering difference is capped at STEER_LIMIT_PCT (60 %, min 40 rpm) of the current speed, so at SLOW the outer wheel never runs faster than 1.6× the set speed; it only matters at slow speeds, where the 650 rpm clamp is about 2× the whole SLOW speed. When the speed mode changes (e.g. SLOW → MEDIUM after a curve), the PID gains blend from the old mode's to the new mode's as the speed actually ramps, so the AGV is never steered with the weak MEDIUM gains while it is still slow; APPROACH always switches at once.

RFID tags and sequences

These tags drive the route: 4 speed zones (0015 = MEDIUM, placed on the straight after a curve), 11 cabang forks (00AA–00B4 = CABANG 1–11) each with its own branch MEDIUM tag (00C1–00CB), a right-track tag (00BA) and a merge, the homepost stop (0000), 1 checkpoint stop and 3 trolley stations. The fork, branch MEDIUM and merge tags (00AA–00B4, 00C1–00CB, 00BB) are placeholders to replace with scanned tags. Tags are ignored while the auto-tune runs. Since 2026-10-28 the RFID driver parses whole reader frames (CRC-checked, frames split across TCP packets kept), like the Kayaba; the old parser lost reads, which at 0.8 m/s meant missed tags. Each tag pass is logged as "RFID 0015 passed: N reads in T s"; "only 1 read" is a warning that the tag was nearly missed — slow down before it, add a second tag with the same ID about 20–30 cm after it, or improve the reader set-up (power, height, tag facing the antenna). MGS1600 markers (opposite-polarity tape beside the track, 15–30 mm from its edge): as an add-on to the RFID tags, a LEFT and RIGHT marker seen at the same time slows HIGH to MEDIUM (sequence marker_both_medium, "marker_side": "both", "requires_speed": ["HIGH"]; at any other speed, or with one marker alone, nothing happens; once per pair, cooldown 3 s; not while a trolley-station approach is armed). Change their actions in the profile to make a marker do something else.

Tag	Sequence	What it does
0014	speed_zone_slow	Speed SLOW, no curve feedforward (this tag is used before both left and right curves; the PID steers)
001C	speed_zone_slow_alt	Speed SLOW (second tag)
0016	speed_zone_high	Speed HIGH
0015	speed_zone_medium	Speed MEDIUM (put it on the straight after a curve)
00AA	fork_to_point1	SLOW + follow RIGHT track — only if CABANG 1 is selected
00AB	fork_to_point2	SLOW + follow RIGHT track — only if CABANG 2 is selected
00AC	fork_to_point3	SLOW + follow RIGHT track — only if CABANG 3 is selected
00AD	fork_to_point4	SLOW + follow RIGHT track — only if CABANG 4 is selected

Tag	Sequence	What it does
00AE	fork_to_point5	SLOW + follow RIGHT track — only if CABANG 5 is selected
00AF, 00B0–00B4	fork_to_point6 … fork_to_point11	SLOW + follow RIGHT track — only if CABANG 6 … 11 is selected
00C1–00CB	branch_medium_point1 … 11	Speed MEDIUM + follow LEFT track on branch 1 … 11 — only if that CABANG is selected (00C1 = CABANG 1 … 00CB = CABANG 11)
00BA	track_right_BA	SLOW + follow RIGHT track (always, no CABANG check)
00BB	fork_merge_revert	Back to LEFT track after the merge (always)
0000	homepost_stop	HOMEPOST: brake, AUTO ends (back to ARMED, run log saved); START / G starts the next trip. Ignored in the first 5 s after START. Put a SLOW tag before it.
000C	checkpoint_stop	Brake and hold 4 s, then continue
00CC	trolley_load_00	APPROACH crawl → stop on PROX (DI8, 45 s timeout) → 1 s → PLC SEQ 0 → wait complete → resume SLOW
0078	trolley_load_01	APPROACH crawl → stop on PROX (30 s) → 1 s → PLC SEQ 1 → wait complete (60 s) → 1 s → resume SLOW
00CD	trolley_load_02	SLOW → stop on PROX (30 s) → PLC SEQ 2 → wait complete (30 s) → resume SLOW

After every trolley station the AGV restarts at SLOW (0.09 m/s) and stays SLOW until a speed tag (e.g. 0016 = HIGH) changes it. If the marker never comes (approach timeout) the AGV carries on at HIGH; if the PLC does not report the conveyor finished in time, the AGV stays stopped until RESET.

A `set_speed` action can use any of the five speed modes: APPROACH, SLOW, MEDIUM, HIGH or MAX. The branch tags (00AA–00AE, 00BA) use `"corner": false`, so they slow down without the left-curve feed-forward of the corner tags.

- **Cabang** is selected on the HOME page (CABANG 1–5, or SEQ 0–4 which also sets it) and is kept through RESET and emergency.
- NAV (speed-zone tags) → SEQ (stops and PLC handshakes) → SLMP (PLC link) can be switched on and off live on the PARAMS page; turning one off turns off the ones after it.
- Add or change a tag in `sequences` in `profiles/agv1_kim.json`; action types are `set_speed`, `set_track`, `sequence_stop`, `wait_marker`, `stop_agv`, `wait_seconds`, `plc_request`, `wait_plc_complete`, `resume`.

Floor magnetic disturbances

Steel or magnetic material under the floor can upset the MGS1600 in two ways. The controller handles both; settings are in profile block `can_sensor`.

Type	What the sensor reports	What the AGV does
A — spot beside the tape	A second "track": left and right more than `TRACK_SPREAD_MM` (15 mm) apart where there is no fork	**Track lock** (currently OFF: TRACK_LOCK_ENABLED false; set true to use it): follows the track nearest the last good reading and ignores the other; log line `[TRACK] second track outside a fork`
B — spot merged with the tape	One wider tape; the position shifts toward the spot	**Sensor hold** (RFID tag): crosses `distance_m` with the steering held and the sensor ignored, tape loss ignored too

- **Fork area:** starts at every `set_track` tag (00AA–00AE when selected, 00BA, 00BB) or K / L key, and ends 0.5 s after the two tracks have come back together (max 60 s). Inside it the AGV follows the chosen side as before, so a spot inside a fork area is not filtered.
- **Fork without a tag:** with track lock the AGV stays on the tape it is on (it no longer takes the left branch by default). `TRACK_LOCK_ENABLED: false` restores the old behaviour.
- **Sensor hold tag:** put an RFID tag just before a type-B spot with the action `{"type": "sensor_hold", "distance_m": 0.5}` (max `SENSOR_HOLD_MAX_M` 1.0 m). Use it on straight tape only: the AGV keeps the steering it had before the spot. Add a `set_speed` SLOW action in the same sequence if the spot is at speed. For a longer area use a pair of tags: straight_start (profile example straight_zone_start, placeholder tag 00D1) makes the AGV drive straight without the sensor, keeping its average steering of the last second, until straight_end (straight_zone_end, placeholder 00D2). If the END

tag is missed it follows the tape again after STRAIGHT_ZONE_MAX_M (3 m). Put the START tag on a straight at least 1 m after a curve, and set the wheel balance first — an unbalanced AGV drifts slowly while blind.

- **HOME page:** TRACK shows `SPOT IGNORED`, `FORK` or `SENSOR HOLD`, plus the number of spots ignored since start; the log has the left / right readings of each spot.
- **Sensor side:** set tape width to 25 mm for 30 mm tape (`^TWDT 0`), check the polarity (`TPOL`), do the zero calibration away from steel (`%ZERO`), save with `%EESAV`, and mount the sensor 20–30 mm above the tape. Double tape over a strong spot also helps.
- The PID auto-tune also ignores a type-A spot (it keeps to the nearest track) and does not stop for it; a type-B spot still spoils the fit, so run it on a clean straight.

Web HMI and API

The HMI has six pages on port 5000, reached from the nav bar at the bottom of each page.

Page	Path	Used for
HOME	`/`	Mode, speed, sensor, RFID, PLC bits, sequence status; set CABANG / SEQ
MANUAL	`/manual`	Jog buttons and arrow keys; ROTATE 180° buttons; WEB BUTTON MODE (virtual START / RESET / MAN-AUTO / EMG)
IO	`/io`	Live DI / DO / AO
TROLLEY	`/trolley`	PLC pusher and conveyor buttons, X / Y lamps (MANUAL only)
PARAMS	`/params`	Feature toggles, live tuning, wheel calibration, PID auto-tune, restart
ERRORS	`/errors`	Recent warnings and errors; RUN LOGS: download each AUTO run's recording (CSV with error, L/R RPM and voltage, P / I / D, KP, speed mode, both sensor tracks, RFID and mode-change events; plus a plot) and the controller log, one by one or the latest 5 as a zip (/api/runlogs/zip). A run is saved when AUTO ends, or at once with SAVE CURRENT RUN

Endpoint	Method	Purpose
`/api/state`	GET	Full AGV state (mode, sensor, PLC, keypad, sequences)
`/api/manual/command`	POST	Jog command (400 ms watchdog), MANUAL only
`/api/keyboard`	POST	One-press keypad action (the 180° turn has its own POST /api/rotate with direction left, right or stop), e.g. `{"key":"enter"}`
`/api/fork_target`	POST	Set CABANG `point_1` – `point_5` or clear
`/api/sequence`	POST	PLC sequence request 0–8 (0–4 also set CABANG 1–5)
`/api/web_button/mode`, `/api/web_button/press`	POST	Web button bypass of the panel buttons
`/api/params`, `/api/params/features`	GET / POST	Profile values; NAV / SEQ / SLMP live toggles
`/api/params/tuning`, `/api/params/tuning/reset`	GET / POST	Live tuning (saved), reset to profile
`/api/autotune`, `/start`, `/stop`	GET / POST	PID auto-tune status and control
`/api/calibration/status`, `/start`, `/stop`	GET / POST	Wheel-speed calibration ramp
`/api/trolley/state`, `/api/trolley/write`	GET / POST	PLC trolley bits
`/api/io`, `/api/errors`, `/api/wifi`	GET	IO table, error log, Wi-Fi signal
`/api/restart`	POST	Restart `agv-controller.service`

PID auto-tune and live tuning

The auto-tune on the PARAMS page measures how the AGV steers on a straight tape and proposes KP / TD / N for MAX, HIGH, MEDIUM, SLOW or APPROACH; APPLY saves them to `data/tuning_agv1_kim.json`, which overrides the profile after a restart.

1. Put the AGV in ARMED on a straight tape with clear space ahead (about speed × time limit metres) of single tape.
2. PARAMS → PID AUTO-TUNE: choose MAX, HIGH, MEDIUM, SLOW or APPROACH, keep Relay 20 rpm, Max error 50 mm, Time limit 30 s, Cycles 4, press START.
3. The AGV accelerates on its current gains, weaves gently for up to 10 s (relay test), then brakes and fits a model.
4. Three gain sets appear: Calm, Recommended, Responsive. Press APPLY on one, then check with a normal AUTO run.

- If it says the relay never steered back, raise Relay (rpm) and use a straighter section. At slow speeds (SLOW, APPROACH) the proposed N is lowered so 1 mm of sensor noise moves a wheel by at most 10 % of the speed — with the full N the AGV surged fast / slow at branches, where the reading is noisiest.
- **Off-centre at speed** (e.g. 40–50 mm at MAX): one wheel runs faster, and the integral term cannot fix it above its 20 mm deadband. The auto-tune measures it and shows a **Wheel balance** row with APPLY — even when warm-up stops on the error limit. Apply it, then run the auto-tune again for the gains; the second run refines the balance. The same side-pull makes the AGV drift off the tape when it speeds up from SLOW to MEDIUM after a curve while MEDIUM still has the default gains (a copy of HIGH): set the balance, then auto-tune MEDIUM.
- APPROACH: run it on straight tape WITHOUT the guide bar, which would hold the AGV and spoil the fit. Only KP / TD / N change; the +4 rpm into-bar and −40 rpm away clamps stay. The proposals will be much higher than the current KP_APPROACH 0.225, so start with Calm and check a station stop.
- RESET, STOP, E, emergency, MANUAL, tape loss or sensor timeout end the test and brake; RFID tags are ignored during it. The test does not stop at a branch or floor spot: it follows the left / right track like before (with TRACK_LOCK_ENABLED on it would keep to the nearest track instead).
- LIVE TUNING on the same page edits speeds, ramps, gains, clamps, wheel balance and feedforward; changes are saved and kept after restart. RESET TO PROFILE deletes the saved file. Tuning is locked while RUNNING, in emergency, or during the auto-tune.
- The measured model uses `kinematics.SENSOR_LOOKAHEAD_M` (axle to front sensor); 0.076 m is a placeholder until measured.

Updating code and troubleshooting

Code changes go laptop → GitHub → AGV: commit and **Push origin** in GitHub Desktop, then on the AGV `cd ~/agv-kim2a-controller && git pull` and restart the controller (`sudo systemctl restart agv-controller.service`, or the RESTART button on

PARAMS). Without internet on the AGV, push over SSH instead: `git push agv main` from the laptop (remote `agv` = `agv1-kim@<AGV address>:/home/agv1-kim/agv-kim2a-controller`).

Symptom	Check
Keypad does nothing	`groups` must include `input`; `python3 debugging/list_keypads.py` must list "XING WEI"; the controller must be running; `/api/state` → `keypad_devices`
W/A/S/D don't move	AGV must be in MANUAL (press Q); EMERGENCY needs ENTER/RESET
MANUAL forward drifts right (or left)	One wheel runs faster. PARAMS → LIVE TUNING → WHEEL BALANCE: raise it (e.g. +2, +3) for a pull to the RIGHT, below 0 for a pull to the LEFT; drive straight and repeat. It applies in MANUAL and AUTO and is saved. Large values (> 5 %) point to a wheel, tyre or brake problem
W + A spins instead of curving	Keypad may not send two keys at once — test with `python3 -m evdev.evtest`
START ignored	Tape not under the front sensor (HOME page TAPE lamp)
AGV stops with "LOST TAPE" / "CAN TIMEOUT"	Sensor off the tape, or CAN adapter / MGS1600 link down
SYSTEM ERROR	A driver stopped answering (DI module, CAN or RFID); check cables and the IPs above
PLC keys / TROLLEY buttons do nothing	MANUAL only; SLMP must be ON (PARAMS); PLC at 192.168.4.10 reachable
Fork tag ignored	No CABANG selected, or a different one selected; placeholder tag hex not yet replaced
SSH "host identification has changed"	Address reused by another machine — `ssh-keygen -R <address>` and verify the fingerprint
`git push agv` refused	Files changed on the AGV — commit them there first, then pull on the laptop

Open items: measure `SENSOR_LOOKAHEAD_M`; replace placeholder tags 00AA–00AE and 00BB; fix `agv2_kim.json` (missing `DECEL_RATE`); give the AGV a fixed IP on the Wi-Fi.